

Lab_6_CLIPDiffusion_without_fMRI

November 10, 2025

1 Contrastive learning with text prompts with CLIP guided diffusion (without fMRI).

CLIP, or Contrastive Language-Image Pretraining, is an algorithm developed by OpenAI that understands images and text together. It uses a neural network trained on a large dataset of image-text pairs, learning to associate images with their corresponding textual descriptions.

The uniqueness of CLIP lies in its ability to comprehend and connect the semantics of images and text without relying on explicit labels.

This notebook uses OpenAI's 512x512 class-conditional ImageNet diffusion model (<https://github.com/openai/guided-diffusion>) together with CLIP (<https://github.com/openai/CLIP>) to connect text prompts with images.

Note: This notebook requires 16 GB of GPU memory to work, if you are unable to get a 16 GB GPU consistently, try the [256x256 version](#).

```
[1]: # Check the GPU
```

```
!nvidia-smi
```

Mon Nov 10 09:05:57 2025

```
+-----+
+-----+
| NVIDIA-SMI 550.54.15                Driver Version: 550.54.15          CUDA Version:
12.4    |
|-----+-----+-----+-----+
+-----+
| GPU  Name                       Persistence-M | Bus-Id        Disp.A | Volatile
Uncorr. ECC |
| Fan  Temp   Perf           Pwr:Usage/Cap |      Memory-Usage | GPU-Util
Compute M.  |
|              |                          |                      |
MIG M.       |
|=====+=====+=====+=====|
=====|
|    0   Tesla T4                       Off | 00000000:00:04.0 Off |
0 |
| N/A    53C    P8              10W / 70W |      0MiB / 15360MiB |      0%
Default |
```

```

|                                     |                                     |
N/A |
+-----+-----+-----+
-----+

+-----+
-----+
| Processes:
|
| GPU    GI    CI          PID    Type    Process name
GPU Memory |
|          ID    ID
Usage      |
|=====
=====|
| No running processes found
|
+-----+
-----+

```

[3]: *# Install dependencies*

```

!git clone https://github.com/openai/CLIP
!git clone https://github.com/crowsonkb/guided-diffusion
!pip install -e ./CLIP
!pip install -e ./guided-diffusion

```

```

fatal: destination path 'CLIP' already exists and is not an empty directory.
fatal: destination path 'guided-diffusion' already exists and is not an empty
directory.
Obtaining file:///content/CLIP
  Preparing metadata (setup.py) ... done
Requirement already satisfied: ftfy in /usr/local/lib/python3.12/dist-packages
(from clip==1.0) (6.3.1)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-
packages (from clip==1.0) (25.0)
Requirement already satisfied: regex in /usr/local/lib/python3.12/dist-packages
(from clip==1.0) (2024.11.6)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages
(from clip==1.0) (4.67.1)
Requirement already satisfied: torch in /usr/local/lib/python3.12/dist-packages
(from clip==1.0) (2.8.0+cu126)
Requirement already satisfied: torchvision in /usr/local/lib/python3.12/dist-
packages (from clip==1.0) (0.23.0+cu126)
Requirement already satisfied: wcwidth in /usr/local/lib/python3.12/dist-
packages (from ftfy->clip==1.0) (0.2.14)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-
packages (from torch->clip==1.0) (3.20.0)

```

Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.12/dist-packages (from torch->clip==1.0) (4.15.0)

Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch->clip==1.0) (75.2.0)

Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch->clip==1.0) (1.13.3)

Requirement already satisfied: networkx in /usr/local/lib/python3.12/dist-packages (from torch->clip==1.0) (3.5)

Requirement already satisfied: Jinja2 in /usr/local/lib/python3.12/dist-packages (from torch->clip==1.0) (3.1.6)

Requirement already satisfied: fsspec in /usr/local/lib/python3.12/dist-packages (from torch->clip==1.0) (2025.3.0)

Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.6.77 in /usr/local/lib/python3.12/dist-packages (from torch->clip==1.0) (12.6.77)

Requirement already satisfied: nvidia-cuda-runtime-cu12==12.6.77 in /usr/local/lib/python3.12/dist-packages (from torch->clip==1.0) (12.6.77)

Requirement already satisfied: nvidia-cuda-cupti-cu12==12.6.80 in /usr/local/lib/python3.12/dist-packages (from torch->clip==1.0) (12.6.80)

Requirement already satisfied: nvidia-cudnn-cu12==9.10.2.21 in /usr/local/lib/python3.12/dist-packages (from torch->clip==1.0) (9.10.2.21)

Requirement already satisfied: nvidia-cublas-cu12==12.6.4.1 in /usr/local/lib/python3.12/dist-packages (from torch->clip==1.0) (12.6.4.1)

Requirement already satisfied: nvidia-cufft-cu12==11.3.0.4 in /usr/local/lib/python3.12/dist-packages (from torch->clip==1.0) (11.3.0.4)

Requirement already satisfied: nvidia-curand-cu12==10.3.7.77 in /usr/local/lib/python3.12/dist-packages (from torch->clip==1.0) (10.3.7.77)

Requirement already satisfied: nvidia-cusolver-cu12==11.7.1.2 in /usr/local/lib/python3.12/dist-packages (from torch->clip==1.0) (11.7.1.2)

Requirement already satisfied: nvidia-cusparse-cu12==12.5.4.2 in /usr/local/lib/python3.12/dist-packages (from torch->clip==1.0) (12.5.4.2)

Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in /usr/local/lib/python3.12/dist-packages (from torch->clip==1.0) (0.7.1)

Requirement already satisfied: nvidia-nccl-cu12==2.27.3 in /usr/local/lib/python3.12/dist-packages (from torch->clip==1.0) (2.27.3)

Requirement already satisfied: nvidia-nvtx-cu12==12.6.77 in /usr/local/lib/python3.12/dist-packages (from torch->clip==1.0) (12.6.77)

Requirement already satisfied: nvidia-nvjitlink-cu12==12.6.85 in /usr/local/lib/python3.12/dist-packages (from torch->clip==1.0) (12.6.85)

Requirement already satisfied: nvidia-cufile-cu12==1.11.1.6 in /usr/local/lib/python3.12/dist-packages (from torch->clip==1.0) (1.11.1.6)

Requirement already satisfied: triton==3.4.0 in /usr/local/lib/python3.12/dist-packages (from torch->clip==1.0) (3.4.0)

Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from torchvision->clip==1.0) (2.0.2)

Requirement already satisfied: pillow!=8.3.*,>=5.3.0 in /usr/local/lib/python3.12/dist-packages (from torchvision->clip==1.0) (11.3.0)

Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch->clip==1.0)

```

(1.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in
/usr/local/lib/python3.12/dist-packages (from jinja2->torch->clip==1.0) (3.0.3)
Installing collected packages: clip
  Attempting uninstall: clip
    Found existing installation: clip 1.0
    Uninstalling clip-1.0:
      Successfully uninstalled clip-1.0
  Running setup.py develop for clip
Successfully installed clip-1.0
Obtaining file:///content/guided-diffusion
  Preparing metadata (setup.py) ... done
Requirement already satisfied: blobfile>=1.0.5 in
/usr/local/lib/python3.12/dist-packages (from guided-diffusion==0.0.0) (3.1.0)
Requirement already satisfied: torch in /usr/local/lib/python3.12/dist-packages
(from guided-diffusion==0.0.0) (2.8.0+cu126)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages
(from guided-diffusion==0.0.0) (4.67.1)
Requirement already satisfied: pycryptodomex>=3.8 in
/usr/local/lib/python3.12/dist-packages (from blobfile>=1.0.5->guided-
diffusion==0.0.0) (3.23.0)
Requirement already satisfied: urllib3<3,>=1.25.3 in
/usr/local/lib/python3.12/dist-packages (from blobfile>=1.0.5->guided-
diffusion==0.0.0) (2.5.0)
Requirement already satisfied: lxml>=4.9 in /usr/local/lib/python3.12/dist-
packages (from blobfile>=1.0.5->guided-diffusion==0.0.0) (5.4.0)
Requirement already satisfied: filelock>=3.0 in /usr/local/lib/python3.12/dist-
packages (from blobfile>=1.0.5->guided-diffusion==0.0.0) (3.20.0)
Requirement already satisfied: typing-extensions>=4.10.0 in
/usr/local/lib/python3.12/dist-packages (from torch->guided-diffusion==0.0.0)
(4.15.0)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-
packages (from torch->guided-diffusion==0.0.0) (75.2.0)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-
packages (from torch->guided-diffusion==0.0.0) (1.13.3)
Requirement already satisfied: networkx in /usr/local/lib/python3.12/dist-
packages (from torch->guided-diffusion==0.0.0) (3.5)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages
(from torch->guided-diffusion==0.0.0) (3.1.6)
Requirement already satisfied: fsspec in /usr/local/lib/python3.12/dist-packages
(from torch->guided-diffusion==0.0.0) (2025.3.0)
Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.6.77 in
/usr/local/lib/python3.12/dist-packages (from torch->guided-diffusion==0.0.0)
(12.6.77)
Requirement already satisfied: nvidia-cuda-runtime-cu12==12.6.77 in
/usr/local/lib/python3.12/dist-packages (from torch->guided-diffusion==0.0.0)
(12.6.77)
Requirement already satisfied: nvidia-cuda-cupti-cu12==12.6.80 in

```

```

/usr/local/lib/python3.12/dist-packages (from torch->guided-diffusion==0.0.0)
(12.6.80)
Requirement already satisfied: nvidia-cudnn-cu12==9.10.2.21 in
/usr/local/lib/python3.12/dist-packages (from torch->guided-diffusion==0.0.0)
(9.10.2.21)
Requirement already satisfied: nvidia-cublas-cu12==12.6.4.1 in
/usr/local/lib/python3.12/dist-packages (from torch->guided-diffusion==0.0.0)
(12.6.4.1)
Requirement already satisfied: nvidia-cufft-cu12==11.3.0.4 in
/usr/local/lib/python3.12/dist-packages (from torch->guided-diffusion==0.0.0)
(11.3.0.4)
Requirement already satisfied: nvidia-curand-cu12==10.3.7.77 in
/usr/local/lib/python3.12/dist-packages (from torch->guided-diffusion==0.0.0)
(10.3.7.77)
Requirement already satisfied: nvidia-cusolver-cu12==11.7.1.2 in
/usr/local/lib/python3.12/dist-packages (from torch->guided-diffusion==0.0.0)
(11.7.1.2)
Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in
/usr/local/lib/python3.12/dist-packages (from torch->guided-diffusion==0.0.0)
(0.7.1)
Requirement already satisfied: nvidia-nccl-cu12==2.27.3 in
/usr/local/lib/python3.12/dist-packages (from torch->guided-diffusion==0.0.0)
(2.27.3)
Requirement already satisfied: nvidia-nvtx-cu12==12.6.77 in
/usr/local/lib/python3.12/dist-packages (from torch->guided-diffusion==0.0.0)
(12.6.77)
Requirement already satisfied: nvidia-nvjitlink-cu12==12.6.85 in
/usr/local/lib/python3.12/dist-packages (from torch->guided-diffusion==0.0.0)
(12.6.85)
Requirement already satisfied: nvidia-cufile-cu12==1.11.1.6 in
/usr/local/lib/python3.12/dist-packages (from torch->guided-diffusion==0.0.0)
(1.11.1.6)
Requirement already satisfied: triton==3.4.0 in /usr/local/lib/python3.12/dist-
packages (from torch->guided-diffusion==0.0.0) (3.4.0)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in
/usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch->guided-
diffusion==0.0.0) (1.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in
/usr/local/lib/python3.12/dist-packages (from jinja2->torch->guided-
diffusion==0.0.0) (3.0.3)
Installing collected packages: guided-diffusion
  Attempting uninstall: guided-diffusion
    Found existing installation: guided-diffusion 0.0.0
    Uninstalling guided-diffusion-0.0.0:
      Successfully uninstalled guided-diffusion-0.0.0

```

Running setup.py develop for guided-diffusion
 Successfully installed guided-diffusion-0.0.0

[5]: *# Download the diffusion model*

```
!curl -OL 'https://openaipublic.blob.core.windows.net/diffusion/jul-2021/
↳512x512_diffusion.pt'
```

% Total	% Received	% Xferd	Average Speed		Time	Time	Time	Current
			Dload	Upload	Total	Spent	Left	Speed
100 2132M	100 2132M	0 0	12.0M	0	0:02:56	0:02:56	--:--:--	11.4M

[6]: *# Imports*

```
import math
import io
import sys

from IPython import display
from PIL import Image
import requests
import torch
from torch import nn
from torch.nn import functional as F
from torchvision import transforms
from torchvision.transforms import functional as TF
from tqdm.notebook import tqdm

sys.path.append('./CLIP')
sys.path.append('./guided-diffusion')

import clip
from guided_diffusion.script_util import create_model_and_diffusion,
↳model_and_diffusion_defaults
```

```
/content/guided-diffusion/guided_diffusion/nn.py:143: FutureWarning:
`torch.cuda.amp.custom_fwd(args...)` is deprecated. Please use
`torch.amp.custom_fwd(args..., device_type='cuda')` instead.
  @th.cuda.amp.custom_fwd
/content/guided-diffusion/guided_diffusion/nn.py:153: FutureWarning:
`torch.cuda.amp.custom_bwd(args...)` is deprecated. Please use
`torch.amp.custom_bwd(args..., device_type='cuda')` instead.
  @th.cuda.amp.custom_bwd
```

[8]: *# Define necessary functions*

```
def fetch(url_or_path):
```

```

    if str(url_or_path).startswith('http://') or str(url_or_path).
↳startswith('https://'):
        r = requests.get(url_or_path)
        r.raise_for_status()
        fd = io.BytesIO()
        fd.write(r.content)
        fd.seek(0)
        return fd
    return open(url_or_path, 'rb')

class MakeCutouts(nn.Module):
    def __init__(self, cut_size, cutn, cut_pow=1.):
        super().__init__()
        self.cut_size = cut_size
        self.cutn = cutn
        self.cut_pow = cut_pow

    def forward(self, input):
        sideY, sideX = input.shape[2:4]
        max_size = min(sideX, sideY)
        min_size = min(sideX, sideY, self.cut_size)
        cutouts = []
        for _ in range(self.cutn):
            size = int(torch.rand([])**self.cut_pow * (max_size - min_size) +
↳min_size)
            offsetx = torch.randint(0, sideX - size + 1, ())
            offsety = torch.randint(0, sideY - size + 1, ())
            cutout = input[:, :, offsety:offsety + size, offsetx:offsetx + size]
            cutouts.append(F.adaptive_avg_pool2d(cutout, self.cut_size))
        return torch.cat(cutouts)

def spherical_dist_loss(x, y):
    x = F.normalize(x, dim=-1)
    y = F.normalize(y, dim=-1)
    return (x - y).norm(dim=-1).div(2).arcsin().pow(2).mul(2)

def tv_loss(input):
    """L2 total variation loss, as in Mahendran et al."""
    input = F.pad(input, (0, 1, 0, 1), 'replicate')
    x_diff = input[..., :-1, 1:] - input[..., :-1, :-1]
    y_diff = input[..., 1:, :-1] - input[..., :-1, :-1]
    return (x_diff**2 + y_diff**2).mean([1, 2, 3])

class MakeCutouts(nn.Module):

```

```

def __init__(self, cut_size, cutn, cut_pow=1.):
    super().__init__()
    self.cut_size = cut_size
    self.cutn = cutn
    self.cut_pow = cut_pow

def forward(self, input):
    sideY, sideX = input.shape[2:4]
    max_size = min(sideX, sideY)
    min_size = min(sideX, sideY, self.cut_size)

    cutouts = []
    for _ in range(self.cutn):
        size = int(torch.rand([])**self.cut_pow * (max_size - min_size) +
min_size)
        offsetx = torch.randint(0, sideX - size + 1, ())
        offsety = torch.randint(0, sideY - size + 1, ())
        cutout = input[:, :, offsety:offsety + size, offsetx:offsetx + size]
        cutouts.append(F.adaptive_avg_pool2d(cutout, self.cut_size))
        #cutouts.append(F.adaptive_max_pool2d(cutout, self.cut_size))

    return torch.cat(cutouts)

```

[9]: *# Model settings*

```

model_config = model_and_diffusion_defaults()
model_config.update({
    'attention_resolutions': '32, 16, 8',
    'class_cond': True,
    'diffusion_steps': 1000,
    'rescale_timesteps': True,
    'timestep_respadding': '1000', # Modify this value to decrease the number of
# timesteps.

    'image_size': 512,
    'learn_sigma': True,
    'noise_schedule': 'linear',
    'num_channels': 256,
    'num_head_channels': 64,
    'num_res_blocks': 2,
    'resblock_updown': True,
    'use_fp16': True,
    'use_scale_shift_norm': True,
})

```

[10]: *# Load models*

```

device = torch.device('cuda:0' if torch.cuda.is_available() else 'cpu')

```



```

print('Using device:', device)

model, diffusion = create_model_and_diffusion(**model_config)
model.load_state_dict(torch.load('512x512_diffusion.pt', map_location='cpu'))
model.requires_grad_(False).eval().to(device)
for name, param in model.named_parameters():
    if 'qkv' in name or 'norm' in name or 'proj' in name:
        param.requires_grad_()
if model_config['use_fp16']:
    model.convert_to_fp16()

clip_model = clip.load('ViT-B/16', jit=False)[0].eval().requires_grad_(False).
    ↪to(device)
clip_size = clip_model.visual.input_resolution
normalize = transforms.Normalize(mean=[0.48145466, 0.4578275, 0.40821073],
                                std=[0.26862954, 0.26130258, 0.27577711])

```

Using device: cuda:0

100% | 335M/335M [00:03<00:00, 111MiB/s]

1.1 Settings for this run:

```

[11]: prompt = 'Futuristic brain in Krakow' #After Midjourney you know how this works!
batch_size = 1
clip_guidance_scale = 1000 # Controls how much the image should look like the
    ↪prompt.
tv_scale = 150 # Controls the smoothness of the final output.
cutn = 40
cut_pow = 0.5
n_batches = 1
init_image = None # This can be an URL or Colab local path and must be in
    ↪quotes.
skip_timesteps = 0 # This needs to be between approx. 200 and 500 when using
    ↪an init image.
# Higher values make the output look more like the init.
seed = 0

```

1.1.1 Actually do the run...

```

[12]: def do_run():
    if seed is not None:
        torch.manual_seed(seed)

    text_embed = clip_model.encode_text(clip.tokenize(prompt).to(device)).
    ↪float()

```

```

init = None
if init_image is not None:
    init = Image.open(fetch(init_image)).convert('RGB')
    init = init.resize((model_config['image_size'],
↳model_config['image_size']), Image.LANCZOS)
    init = TF.to_tensor(init).to(device).unsqueeze(0).mul(2).sub(1)

make_cutouts = MakeCutouts(clip_size, cutn, cut_pow)

cur_t = None

def cond_fn(x, t, y=None):
    with torch.enable_grad():
        x = x.detach().requires_grad_()
        n = x.shape[0]
        my_t = torch.ones([n], device=device, dtype=torch.long) * cur_t
        out = diffusion.p_mean_variance(model, x, my_t,
↳clip_denoised=False, model_kwargs={'y': y})
        fac = diffusion.sqrt_one_minus_alphas_cumprod[cur_t]
        x_in = out['pred_xstart'] * fac + x * (1 - fac)
        clip_in = normalize(make_cutouts(x_in.add(1).div(2)))
        image_embeds = clip_model.encode_image(clip_in).float().view([cutn,
↳n, -1])
        dists = spherical_dist_loss(image_embeds, text_embed.unsqueeze(0))
        losses = dists.mean(0)
        tv_losses = tv_loss(x_in)
        loss = losses.sum() * clip_guidance_scale + tv_losses.sum() *
↳tv_scale
        return -torch.autograd.grad(loss, x)[0]

if model_config['timestep_respacing'].startswith('ddim'):
    sample_fn = diffusion.ddim_sample_loop_progressive
else:
    sample_fn = diffusion.p_sample_loop_progressive

for i in range(n_batches):
    cur_t = diffusion.num_timesteps - skip_timesteps - 1

    samples = sample_fn(
        model,
        (batch_size, 3, model_config['image_size'],
↳model_config['image_size']),
        clip_denoised=False,
        model_kwargs={'y': torch.zeros([batch_size], device=device,
↳dtype=torch.long)},
        cond_fn=cond_fn,

```

```

        progress=True,
        skip_timesteps=skip_timesteps,
        init_image=init,
        randomize_class=True,
    )

    for j, sample in enumerate(samples):
        cur_t -= 1
        if j % 100 == 0 or cur_t == -1:
            print()
            for k, image in enumerate(sample['pred_xstart']):
                filename = f'progress_{i * batch_size + k:05}.png'
                TF.to_pil_image(image.add(1).div(2).clamp(0, 1)).
↪save(filename)

                tqdm.write(f'Batch {i}, step {j}, output {k}:')
                display.display(display.Image(filename))

do_run()

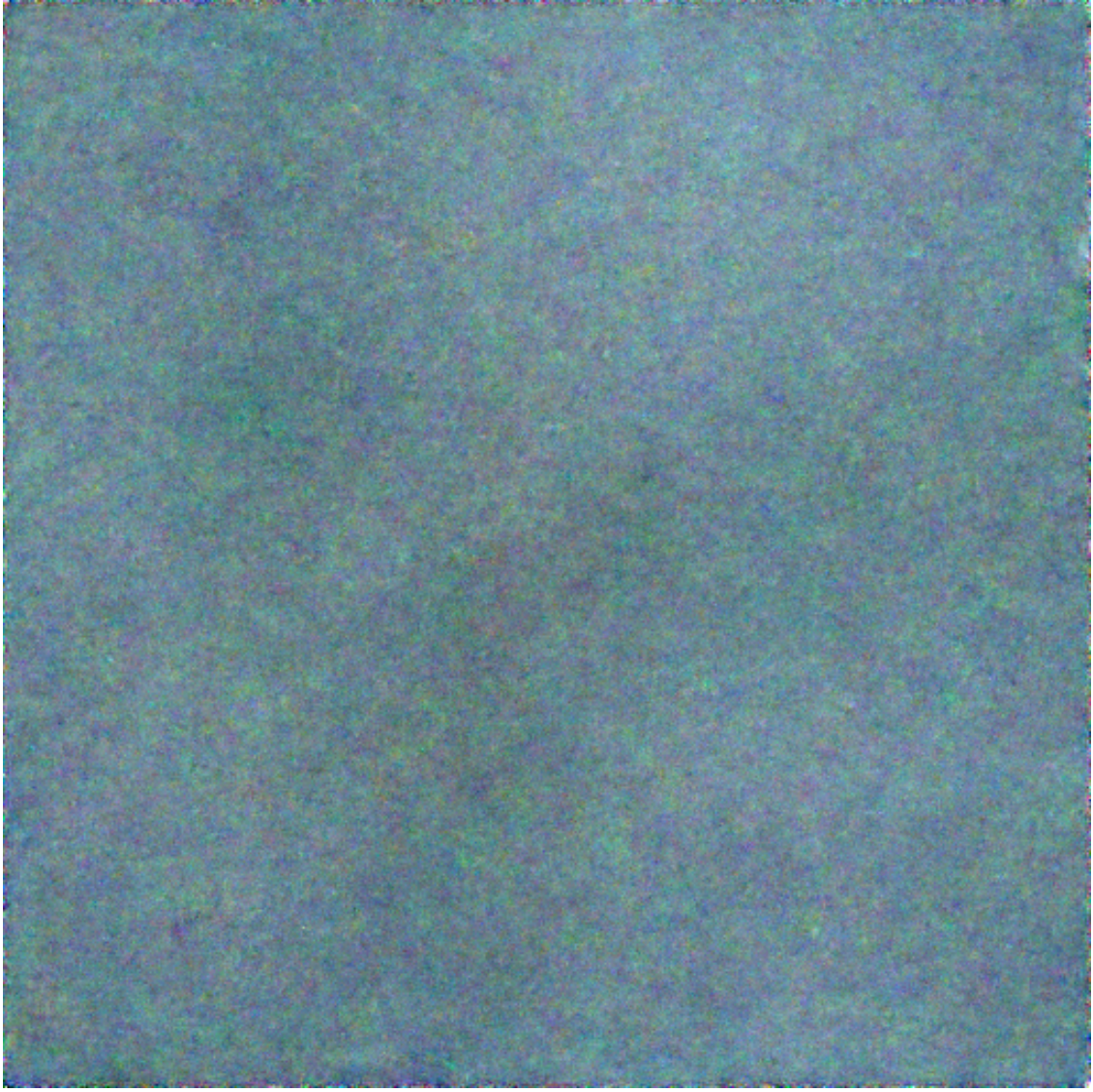
```

```

0%|          | 0/1000 [00:00<?, ?it/s]

```

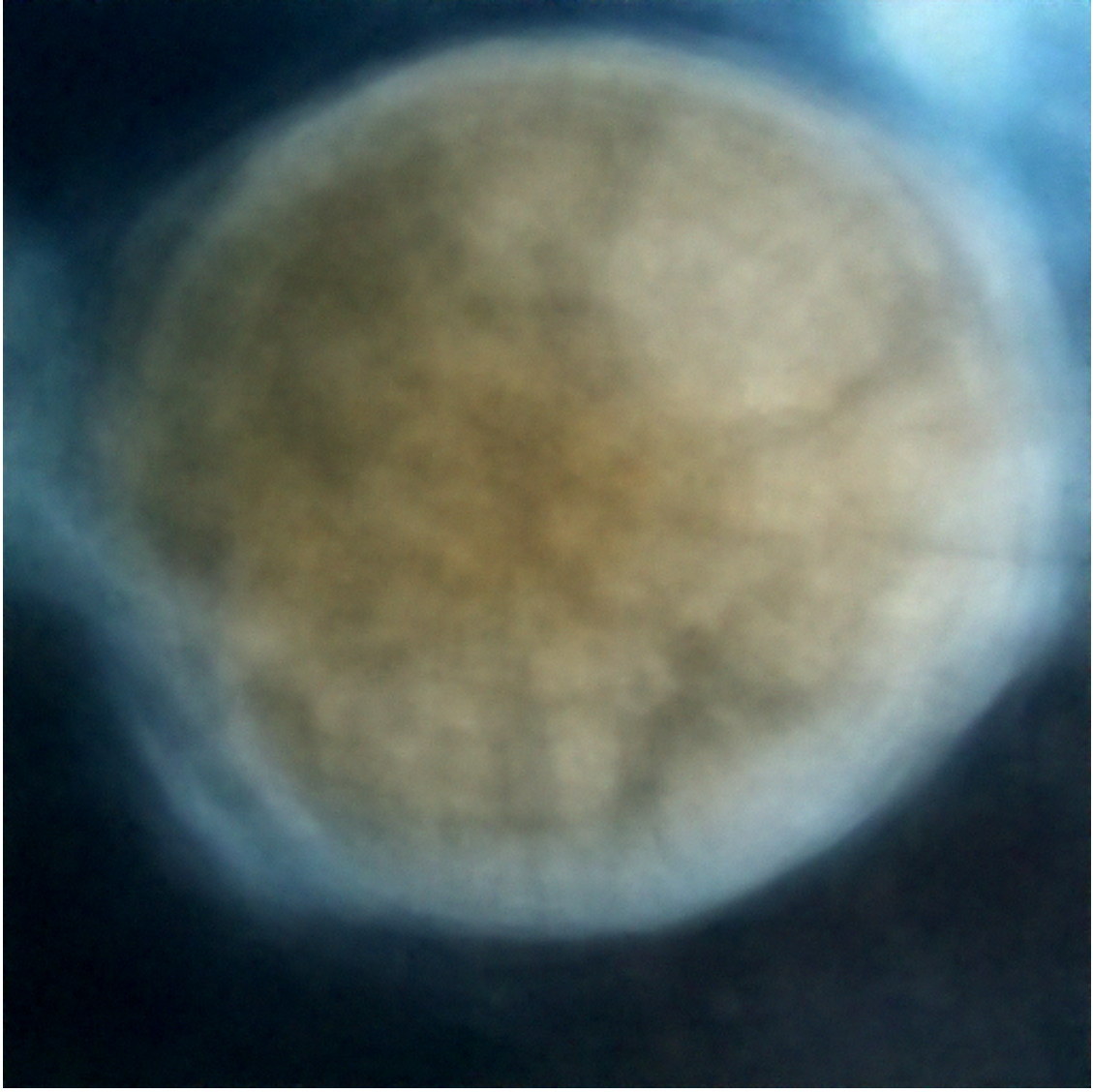
Batch 0, step 0, output 0:



Batch 0, step 100, output 0:



Batch 0, step 200, output 0:



Batch 0, step 300, output 0:



Batch 0, step 400, output 0:



Batch 0, step 500, output 0:



Batch 0, step 600, output 0:



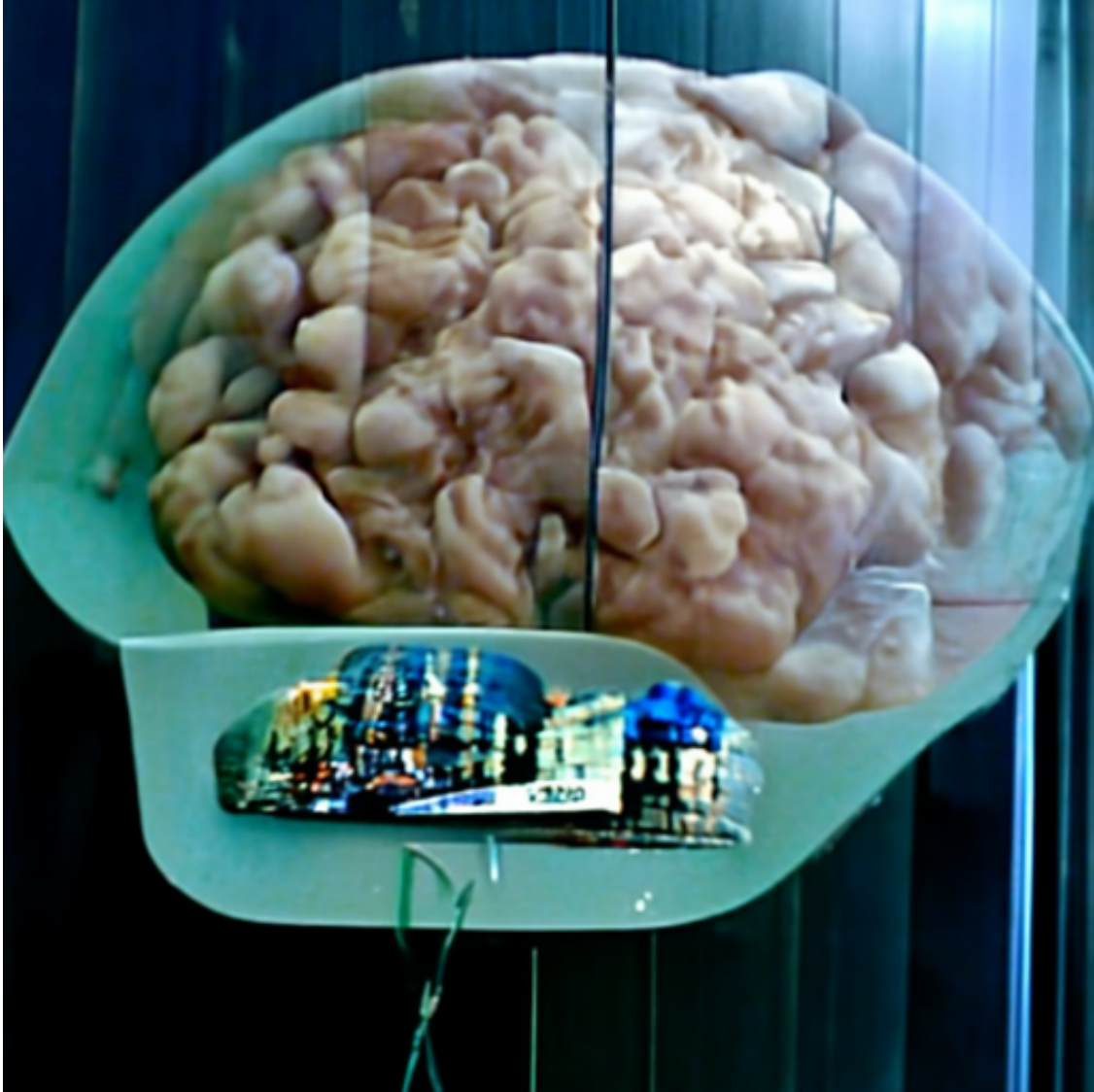
Batch 0, step 700, output 0:



Batch 0, step 800, output 0:



Batch 0, step 900, output 0:



Batch 0, step 999, output 0:

